



A DATA BASE SUBLANGUAGE FOUNDED ON THE RELATIONAL CALCULUS

by

E. F. Codd
IBM Research Laboratory
San Jose, California

ABSTRACT:

Three principal types of language for data base manipulation are identified: the low-level, procedure-oriented (typified by the CODASYL-proposed DML), the intermediate level, algebraic (typified by the Project MAC MacAIMS language), and the high level, relational calculus-based data sublanguage, an example of which is described in this paper. The language description is informal and stresses concepts and principles. Following this, arguments are presented for the superiority of the calculus-based type of data base sublanguage over the algebraic, and for the algebraic over the low-level procedural. These arguments are particularly relevant to the questions of inter-system compatibility and standardization.

1. Introduction

The two main goals of this paper are:

1. To describe a data base sublanguage (ALPHA) founded on the relational model [1,2] and also on the relational calculus [3];
2. To compare this calculus-oriented sublanguage with two other types of sublanguage used or proposed for data base manipulation.

Two reasons for publishing this paper at this time are:

1. It exhibits a high-level approach to data base language which is basically different from that proposed by the CODASYL Data Base Task Group [4];
2. It is intended to provide a framework for subsequent papers on authorization principles, search tactics and data representation techniques.

We use the term data sublanguage (rather than language) because we are not concerned with general processing (or the capability of computing all computable functions). Instead, we wish to focus on only those language components which support storage and retrieval of formatted data from large shared data bases.

Excluded from discussion in this paper is language used for describing the physical representations permitted by a data base system.

1.1 Data Model

In [1] the author discussed a number of data models then in use and introduced a relational model based on normalized relations of assorted degrees (not just binary relations).^{*} Further normalization of this model is pursued in [2]. The principal motivations of the relational data model were (and are):

1. data independence
2. the simplest possible structure consistent with semantic considerations
3. provision of a unifying principle which would simplify a) the language needed for interaction and b) statement analysis needed for authorization of access and optimization of search
4. relatively easy analysis for consistency.

The need for a very high degree of independence (item 1 above) between application programs and terminal activities on the one hand, and the physical (or storage) representation of data on the other hand, is now widely recognized. Two key properties of a system which supports such data independence are:

^{*} For definitions, see Appendix

1. The physical representation of the data may be changed without logically impairing any application programs;
2. The system can automatically translate requests expressed in terms of the logical structure of the data into corresponding operations on whatever physical data structure is currently in force (this automatic translation must continue to work correctly when the physical data structures are changed while the logical data structure is kept fixed).

In some proposed systems (CODASYL [4], SHARE-GUIDE [5]) three levels of data representation are planned: the user logical, the system logical, and the physical. The system logical representation can be thought of as providing the basic data model with which the community of users interacts. Superimposed on this community model of the data is provision for individuals or sub-communities to establish specializations or simple variations of the community model suitable for their particular applications. We shall use the term data model as a concise phrase for the community model of the data.

Existing and proposed data base systems may be classified by the data model with which the user interacts and the language features provided to the user for expressing this interaction. Table 1 displays such a classification.

1.2 Procedural Approach

Most readers of this paper will have some familiarity with the existing systems (and the CODASYL proposal) cited in the top two rows of this table. The data sublanguages for these systems, so far as they can be identified separately* from their host languages, are rather primitive. Not even boolean selection expressions are provided in IDS [9] and CODASYL [4]. Therefore, there is a heavy dependence on the host language for iterative scanning and searching.

Feldman and Rovner [10] introduced a semi-algebraic approach based on binary relations as the data model and user-specified triples (attribute, object, value) as the linguistic means of reference. Ash and Sibley [11] extended this approach.

*The separation in the CODASYL proposal is clear.

DATA MODEL DATA SUBLANGUAGE		TREES	NETS	RELATIONS	
				DEGREE 2 ONLY	ASSORTED DEGREES ($n \geq 1$)
LOW	Minimal Selection Capability		[9] IDS [4] Codasy1		
	Boolean Selection Expressions	[6] IMS [7] GIS [8] UL/1			
INTERMEDIATE	Attribute -Object -Value			[10] Feldman -Rovner [11] Ash -Sibley	
	Algebra Oriented				[15] Childs [16] MacAIMS
HIGH	Calculus Oriented	[20] TDMS			Codd (α)

Table 1: Types of Data Models and Languages

1.3 Algebraic Approach

In the algebraic approach retrieval of data is viewed as the formation of a new relation from the data base relations by use of some operation of the algebra. It should be stressed that these operations act upon entire relations as their operands just as matrix operations treat entire matrices as their operands. Bertrand Russell described several interesting operations on binary relations in the Principia [12].

The algebraic approach has been favored by researchers working in fact retrieval and deductive inference. In addition to their own contributions, Levien and Maron [13] and Bell [14] provide excellent lists of references to work in this area. For our present purposes, let us set aside the heavy emphasis on natural language processing which characterizes much of this work. Typically, in fact retrieval and deductive inference experiments (as well as in many graphics-oriented data bases), there are many relations of low degree and low cardinality (in other words, there are small amounts of data per relation). This probably explains why these researchers restricted their theory and experimentation to binary relations. Fact retrieval researchers also tend to treat their data bases as having no more structure than a collection of sentences (cf. the Rand Relational Data File [13]).

In commercial and industrial data bases which reflect the operations of the company the situation is markedly different. These data bases tend to consist of relatively few relations, often of large degree and large cardinality. The need for a data model in which relations of higher degree than two is permitted is clear. Accordingly, algebras dealing with relations of arbitrary degree are being considered and implemented. Childs was an early contributor, although the operations he lists in [15] are heavily biased towards binary relations.* Goldstein and Strnad [16,17] describe a system called MacAIMS which uses an algebra of normalized relations of arbitrary degree. In [3] this author has defined a relational algebra which possesses as much selective power as the relational calculus.

1.4 Relational Calculus Approach

The Time Shared Data Management System of the System Development Corporation employed a limited form of predicate calculus in its query language. At most, one quantifier was permitted in a query and the interpretation of these queries was highly dependent on the particular tree structuring which a given body of information was cast in.

In [3] the author proposed an applied predicate calculus with tuple variables, subscripts and the concept of range separability. The data sublanguage ALPHA described informally in sections 2 and 3 is based on this calculus.

*For example, in [15] there appears to be no operation which is degree-increasing. An operation on relations is degree-increasing if it produces a relation of higher degree than that of any of its operands. The natural join defined in [1] is an example of a degree-increasing operation.

2. DSL ALPHA: Basic Characteristics

The ALPHA sublanguage is a language for manipulating relations. It possesses the following characteristics:

1. The full power of the relational calculus [3] - an applied predicate calculus with tuple variables;
2. The capability of specifying any of the operations:
 - 1) fetch value or set of values
 - 2) change value or set of values
 - 3) insert element (i.e., tuple) or set thereof into a relation
 - 4) delete element (i.e., tuple) or set thereof from a relation
 - 5) declare a relation and its domains for inclusion in the established set of data base relations
 - 6) drop a relation from the data base.
3. The capability (through RANGE statements) of delimiting the scope of interaction between a user or his program on the one hand and the data base on the other;
4. A variable binding capability to permit use in either an interactive or batch environment.

The language is accordingly more than a query or search language. In this author's opinion the frequent separation of query languages from other aspects of retrieval is usually artificial. The proposed language is intended to be a sublanguage (in a semantic rather than syntactic sense) of the languages used by all terminal users of a shared, formatted data base, except those users who have such narrow needs that their requirements can be more efficiently satisfied by a set of pre-structured (or canned) queries and/or updates.

It is also intended to be a sublanguage of such host programming languages as PL/I, Cobol and Fortran. However, syntactic modifications may be necessary in order to adapt it to these different host languages. Some modification (semantic as well as syntactic) of candidate host languages may be necessary also. For example, it might be desirable to introduce RELATION as a data type in Fortran. All computation of functions is defined in host language statements; all retrieval and storage operations in data sublanguage statements. A data sublanguage statement can, however, contain a use of a function defined in host statements.

Unlike previous proposals, the notion of position in a file, entity set, data set or relation is de-emphasized. Data base relations, in particular, are treated as sets whose ordering, organization and physical representation are not known to the majority of users. These representation attributes are assumed to be subject to change without notice. As will be clear later, a position concept is included in the sublanguage, but is relative to a user-defined ordering only. The user can specify an ordering in which the elements to be retrieved are delivered to a designated

workspace. He can also advise the system of an ordering which exists in data that is waiting in a workspace for transmission into the data base.

2.1 The Workspace*

Data flows into and out of the data base via user workspaces as directed by data sublanguage statements (see Fig. 1). A workspace may consist of space in high speed memory or low speed storage, or, for interactive purposes, may be identified with a terminal (video or hard-copy). A single user or application program may have several workspaces in use concurrently.

The workspace concept on which DSL ALPHA is based serves several purposes. Perhaps the most important is that it provides a data interface between the host language and the data sublanguage. The normal user never applies host language statements directly to data residing in the data base, but instead fetches the desired data into one or more workspaces using data sublanguage statements, and then applies host language statements to the data in these workspaces. Statements of DSL ALPHA may be intermixed with host language statements (see 3.19).

Data transmitted from the data base to a workspace W is placed in W with a special representation which can be handled by such host languages as PL/I and Cobol. This interface representation consists of a rectangular array with the following properties:

- 1) data items are homogeneous within each column;
- 2) different columns may have dissimilar types of data items;
- 3) all rows consist of distinct tuples.

In addition, each column is headed by a workspace attribute name. Thus, at each instant of time, each workspace holds the array representation of one relation or a tuple thereof.

Since representations used in the data base can be entirely different from the interface representation, system-determined extraction and re-ordering operations usually accompany any transmission of data between the data base and the workspaces in either direction.

A relation which has been deposited in a workspace W may be used for retrieval purposes as an extension of the collection of relations in the data base, but only by the user or application program which owns W. This feature of the data sublanguage is particularly helpful in breaking down very complicated queries into a sequence of much simpler ones (see example 3.13). The various steps which must be taken before a relation can "go public" (i.e., be established as a sharable data base relation) are summarized in example 3.31.

*The reader who is familiar with the CODASYL user working area is cautioned not to confuse it with the DSL ALPHA workspace (they are quite different concepts).

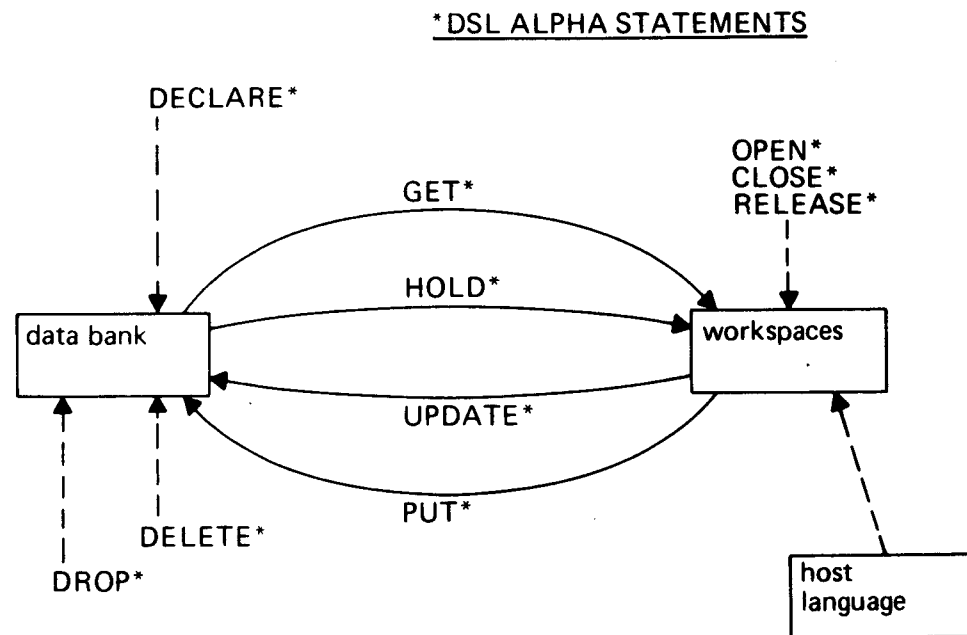


Fig. 1: Information Flow

The user may specify either of two modes of data transmission. In the non-piped mode, data transmission between a workspace and the data base appears to the user to take place all at once. Control is returned to the user only upon completion of transmission of the entire relation involved in the request. In the piped mode data transmission takes place tuple by tuple, and control is returned to the user after each tuple is transmitted (see section 3.18, 3.22, 3.27).

A state diagram is given in Fig. 2 showing permissible changes of state for any workspace. Note that whenever a workspace is in one of the 8 busy states it is inaccessible to statements of both the host language and the data sublanguage. In any of the non-busy states host language statements may be applied to the workspace. Which DSL ALPHA statements are applicable depends (as indicated in Fig. 2) on the particular non-busy state which a workspace is in.

2.2 Names

Basically, the ALPHA sublanguage uses only the following kinds of names: domain names, data base relation names, attribute names, workspace names and tuple variables. Names for data base domains, relations, and attributes are chosen by the community of users (possibly through an agent called the data base administrator). Names for workspaces and tuple variables are chosen by individual users.

The underlying domains, on which the data base relations are defined, must be assigned names which are unique relative to one another. The data base relations must also be named uniquely relative to themselves and to the underlying domains.

When a relation R is defined on certain underlying domains $D_1, D_2, \dots, D_n$, each of these n domain applications is assigned a distinct attribute name (remember that two or more of the domains used to define R may be identical, and we do not want users to be forced to remember domain positions). A relation of degree n thus has n distinct attribute names. If all n domains on which that relation is defined are distinct, then the attribute names can be precisely the same as the domain names. Two applications of a common underlying domain to a relation R are given distinct attribute names by attaching distinct prefixes (called role names in [1]) to a common stem which is the name of the common domain.

For example, suppose one of the underlying domains is a set of serial numbers of parts and the domain name is $P\#$. Suppose there is a relation COMPONENT with attributes named $SUB_P\#$, $SUP_P\#$. Then, the common stem $P\#$ indicates that values for these two domains are always serial numbers of parts. The prefixes SUB, SUP serve to distinguish the two distinct applications of the domain $P\#$ within the single relation COMPONENT. If an attribute of some other relation has a prefixed or non-prefixed name with the stem $P\#$, its values must also come from this same underlying domain.

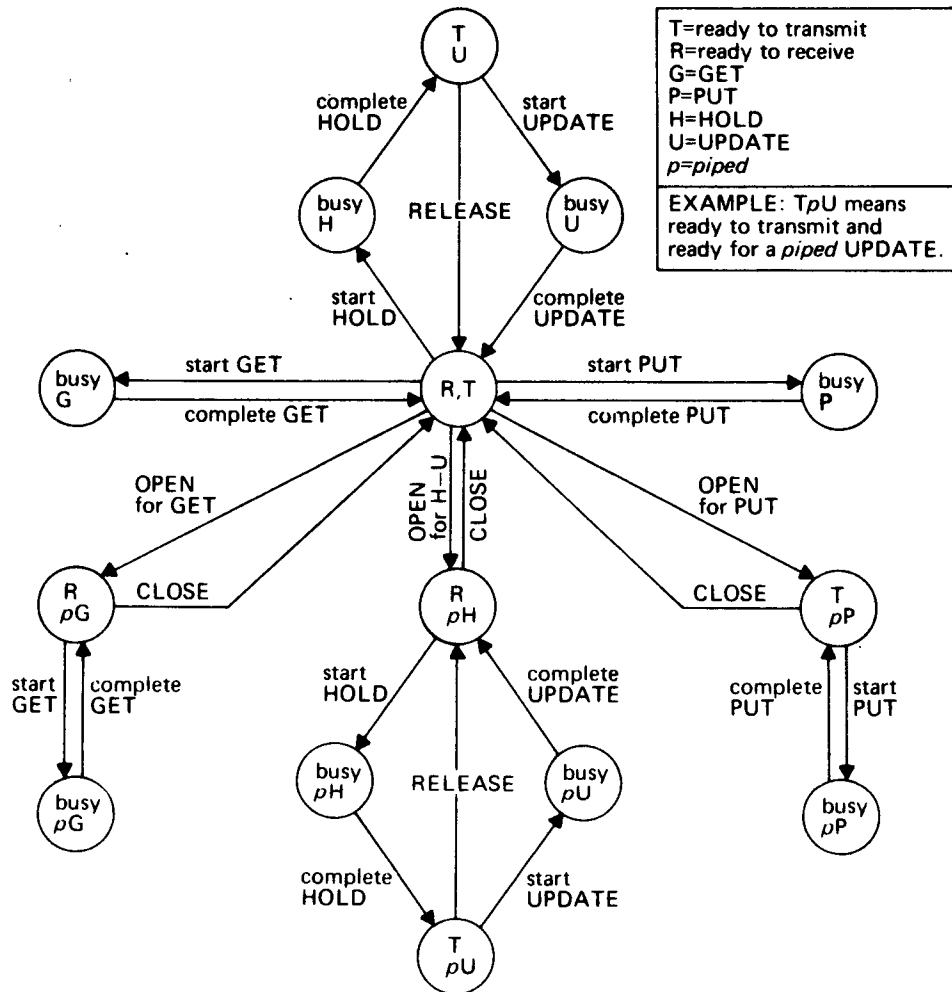


Fig. 2: Transmission Control: State Transition Diagram

We shall omit consideration of user-chosen synonyms for data base domains, relations and attributes, and accordingly assume that each user avoids the use of any data base domain name, relation name, or attribute name as a workspace name or tuple variable. In all other respects, however, each user need not be aware of names in use by other users.

Now, to refer to a particular attribute D in some chosen relation R, we use the qualified name R.D. Similarly, if Z is a tuple variable with range R and we wish to refer to the D component of Z, we use the qualified name Z.D. As examples we might have as qualified names:

SUPPLIER.LOC S.LOC
COMPONENT.SUB_PART C.SUB_PART

where SUPPLIER, COMPONENT are relation names
 LOC, PART are domain names
 LOC, SUB_PART are attribute names
 S is a tuple variable with range SUPPLIER
 C is a tuple variable with range COMPONENT.

One important advantage of having attribute names structured in this way is that they permit systematic handling of the all-occurrence transformations which arise in practice. An example of this is a sweeping change of all part numbers wherever they occur, or even of just one part number wherever it occurs. By examining the catalog of relation names, attribute names (prefixed as necessary) and domain names, the system can discover all of the attributes involving part numbers (see example 3.22).

Domain and attribute names for workspaces receiving information from the data base are system-determined according to a name inheritance rule (see examples 3.2, 3.12, 3.14).

2.3 Absent and irrelevant values

In any practical data base there is a strong likelihood that some value or values are not presently available, while others never will be available because they are (for some reason) irrelevant. Of all the examples given in section 3, only example 3.25 touches upon this subject. The ramifications, however, are substantial.

First of all, a three-valued logic is needed, one which involves the truth values

TRUE, FALSE, MAYBE.

An explanation can be found in [18]. Secondly, when the final evaluation of a condition for eliciting a data item is completed and the result is MAYBE, the question arises: Should this item be included in

the retrieved set or not? In DSL ALPHA (in contrast to [18]), inclusion or exclusion of such items is under the control of the user through his attachment or non-attachment of the keyword MAYBE_TOO on the pertinent retrieval request.

3. Data Sublanguage ALPHA Examples

The examples which follow are intended to introduce the reader gradually to the concepts and semantics of ALPHA. The author does not attach any special importance to the syntax of ALPHA (note for example the absence of an end-of-statement symbol). Examples 1-18 deal with queries, 19-23 with updates, 24-27 with insertions, 28-30 with deletions, 31-32 with establishing and dropping data base relations.

The sample data base which these examples exploit, consists of four relations as tabulated in Table 2. The primary key of each relation is underlined.

<u>Relation</u>	<u>Attributes/Domains</u>
SUPPLIER	<u>S#</u> , SNAME, LOC
PART	<u>P#</u> , PNAME, COLOR, WEIGHT, QOH
PROJECT	<u>J#</u> , JNAME, MGR E#
SUPPLY	<u>S#</u> , <u>P#</u> , <u>J#</u> , SHIPDATE, QUANTITY

Table 2: Sample Data Base

The relation SUPPLY satisfies the following integrity constraints:

1. The set of supplier numbers appearing in the SUPPLY relation is a subset of the set of supplier numbers appearing in the SUPPLIER relation;
2. A similar constraint applies to part numbers relative to the PART relation;
3. A similar constraint applies to project numbers relative to the PROJECT relation.

3.1 Simple Query

** Find all the part numbers of parts being supplied

GET W SUPPLY.P#

The set of distinct part numbers appearing in the part number attribute (P#) of the relation SUPPLY is copied into workspace W. Note that redundant duplicate values of P# are not delivered to W. After execution of this statement, W may be treated as a unary relation, whose sole attribute is P#.

This query has three principal parts:

<u>an operation name</u>	<u>GET</u>
<u>a workspace name</u>	W
<u>a target list</u>	SUPPLY.P#

Generally, a target list is composed of elements each of which is of one of the forms:

relation name.attribute name
tuple variable.attribute name
relation name
tuple variable

In the last two cases, the omission of an attribute name indicates that complete tuples are to be elicited from the corresponding relation.

Because a desired ordering is not specified in this sample query, the part numbers are delivered to W in a system-determined order, which is not guaranteed to be invariant with time.

3.2 Query with Qualification

** Find the part numbers, names and quantities on hand where quantity on hand is less than 25

GET W (PART.P#,PART.PNAME,PART.QOH):(PART.QOH<25)

This statement contains an additional principal part, the qualification. This appears after the target list and, for clarity, is separated from the target list by a colon which can be taken to mean where or such that.

The set of distinct PART triples (P#,PNAME,QOH) satisfying the requirement that the value of quantity-on-hand (QOH) is less than 25 is copied into workspace W. Accordingly, W may be regarded as a normal ternary relation with attributes P#, PNAME,QOH.

A convention called the name inheritance rule permits these attributes to be referred to by the terms

W.P#,W.PNAME,W.QOH

Examples 3.12, 3.13 deal further with this matter.

3.3 Tuple Variables and their Range

Suppose a user is beginning a terminal session or writing an application

program in which he is going to make frequent reference to the relation PART. He can use a range declaration* of the form

RANGE PART P

to tell the system that he wants to talk about parts and is going to use P to designate a typical tuple of the relation PART. This declaration stays in effect until explicitly overridden by another RANGE statement involving P or until termination of the session or until exit of control from the pertinent application program block.

Assuming the above declaration is in effect, example 3.2 can be expressed as follows:

GET W (P.P#,P.PNAME,P.QOH):(P.QOH<25)

Looking back at examples 3.1 and 3.2, we now see that we used a relation name (PART) in those queries to denote not the relation itself, but a variable whose range is that relation. This is permitted in DSL ALPHA, only when neither of the following conditions holds:

1. The variable in question is quantified (see examples 3.6, 3.9);
2. The variable in question shares its range with some other variable (see example 3.7).

One important reason for insisting on RANGE statements is that it forces the user to state the context of his requests, and thereby tends to make it difficult for him to ask unreasonable questions such as: find all the items in the data base that are not suppliers of the part with part number 3. The response to such a query would be almost the entire data base (see [3] for the underlying concept of a range-separable formula of the relational calculus). Incidentally, range declarations do not jeopardize the use of DSL ALPHA in an interpretive, interactive mode.

3.4 User-specified Ordering

** Same query as 3.3, except that elements are to be delivered to the workspace in order by increasing WEIGHT (major order) and decreasing QOH (minor order).

GET W (P.P#,P.PNAME,P.QOH):(P.QOH<25) UP P.WEIGHT
DOWN P.QOH

This statement contains an additional principal part, the element ordering expression. This expression follows the qualification part (if this is null, it follows the colon). It is a list of pairs, each consisting of the keyword UP or DOWN followed by a relation name, period, and attribute name.

* In a somewhat different context E. S. Lowry proposed range declarations (see [19]).

3.5 Limited Number of Elements

- ** Find the part numbers, part names, and quantities on hand where quantity on hand is less than 25, retrieving no more than five elements.

GET W (5)(P.P#,P.PNAME,P.QOH):(P.QOH<25)

Another principal part, the quota, is introduced in this example. It is either an integer or an integer valued expression, and indicates to the system that no more than that particular number of elements are to be delivered to the specified workspace. If no element ordering expression is specified and the data base contains more elements satisfying the specified qualification than the quota, the system determines which ones of the qualified elements are to be delivered. By including an element ordering expression along with a quota, the user can exert partial (and in some cases, complete) control over which elements are delivered.

3.6 Single Existential Quantifier

Consider two queries concerning suppliers and parts. Assume that the following range declarations are in effect.

RANGE SUPPLIER S

RANGE SUPPLY Z

- ** Find the supplier numbers of those suppliers who supply the part with part number 3.

GET W Z.S#:(Z.P#=3)

- ** Find the supplier names of those suppliers who supply the part with part number 3.

GET W S.SNAME: $\exists Z((S.S\#=Z.S\#)\wedge(Z.P\#=3))$

Note that reference must be made to both the SUPPLIER relation and the SUPPLY relation to answer the second query, whereas the SUPPLY relation alone is adequate for the first.

The second GET statement above may be paraphrased as follows: Find the name of every supplier such that there exists a supply tuple having the same supplier number together with an associated part number of value 3. Quantifiers may appear in the qualification, but not in the target list or in the element ordering expression. Where a quantifier appears at the extreme left of a qualification, it may be moved up to the RANGE statement for the variable which is quantified. Thus, the second query above may be expressed as follows:

```

RANGE    SUPPLIER  S
RANGE    SUPPLY    Z SOME
GET  W  S.SNAME:(S.S#=Z.S#)^(Z.P#=3)

```

The attribute SOME indicates existential quantification of the associated tuple variable, while ALL indicates universal quantification. A natural default rule for a missing quantifier attribute on a tuple variable (a variable not appearing in the target list) is that SOME be assumed. All tuple variables appearing in the qualification and not in the target list must be quantified either explicitly or by default. The case of multiple quantifiers and their attachment to RANGE statements is dealt with in example 3.10.

Expressions of the form

```
T.D < U.E, T.D = U.E, T.D > U.E
```

where T,U are tuple variables

D,E are attribute names

are called join terms, since they are the relational calculus counterparts of the relational algebraic operations lo-join, equi-join, and hi-join (see [3]). Such expressions are permitted in DSL ALPHA only in those cases where the units of the attribute T.D are system-convertible into the units of U.E -- for example, T.D could be in grams and U.E in pounds, if the system had a conversion algorithm it could invoke. A common special case of this occurs when T.D and U.E have the same units. For example, both attributes might refer to part numbers or both to ages in years.

Since all qualifications can be converted to a logically equivalent formula with the quantifiers at the extreme left (prenex normal form), it is always possible to express a given query using the attributes SOME or ALL on RANGE statements, so that no quantifiers appear explicitly in the GET statement.

3.7 Multiple Tuple Variables with Common Range

** Find the supplier numbers of those suppliers who have the same location (LOC) as supplier Jones.

```

RANGE    SUPPLIER  S
RANGE    SUPPLIER  T SOME
GET  W  S.S#:(T.SNAME='JONES')^(T.LOC=S.LOC)

```

Note that S and T have the common range SUPPLIER but must be distinguished in the query. The response to this query includes the

supplier number of Jones himself assuming he is recorded in the data base as a supplier.

3.8 Negation

** Find the supplier numbers of suppliers who supply a part other than part number 3.

RANGE SUPPLY Z
GET W Z.S#: $\neg(Z.P\#=3)$

3.9 Single Universal Quantifier

** Find the supplier numbers of suppliers who do not supply part number 3 (and may not supply any part at all at this time).

GET W S.S#: $\neg\exists Z((S.S\#=Z.S\#)\wedge(Z.P\#=3))$

The following statement has a logically equivalent qualification and makes use of the universal quantifier:

GET W S.S#: $\forall Z\neg((S.S\#=Z.S\#)\wedge(Z.P\#=3))$

Because the second version is in prenex normal form (quantifiers at extreme left of the qualification), it is possible in this case to pre-declare the quantification as follows:

RANGE SUPPLIER S
RANGE SUPPLY Z ALL
GET W S.S#: $\neg((S.S\#=Z.S\#)\wedge(Z.P\#=3))$

3.10 Multiple Quantifiers

** Find the names and locations of all suppliers, each of whom supplies all projects.

GET W (S.NAME,S.LOC): $\forall J\exists Z((S.S\#=Z.S\#)\wedge(Z.J\#=J.J\#))$

In this statement the ordering of quantifiers cannot be changed without changing its meaning. Thus, when the quantifiers are moved to the corresponding RANGE statements, these statements must appear in the order shown below:

```

RANGE    SUPPLIER S
RANGE    PROJECT J ALL
RANGE    SUPPLY Z SOME
GET W (S.SNAME,S.LOC):(S.S#=Z.S#)^(Z.J#=J.J#)

```

Note that tuple variables appearing in the target list (in this example, just S) are never quantified.

3.11 Multiple Relation Target List

** For each project obtain as a triple the project number, project name and supplier location for all suppliers who supply that project.

```

RANGE    PROJECT J
RANGE    SUPPLIER S
RANGE    SUPPLY Z SOME
GET W (J.J#,J.JNAME,S.LOC):(J.J#=Z.J#)^(Z.S#=S.S#)

```

The first and second components of each triple copied into W come from the PROJECT relation. The third component comes from the SUPPLIER relation. In general, any number of distinct relations may be referenced in the target list. As usual, any tuple variable appearing in the qualification and not in the target list must be quantified either explicitly or by default.

3.12 Naming of Workspace Attributes

The name inheritance rule mentioned in example 3.2 needs to be re-examined in the light of example 3.11. This rule must be so defined that it is never possible for two distinct attributes of a workspace to inherit identical names. Recall that for each established data base relation it is decreed that its attributes be distinctly named -- if necessary through prefixing as in

```
COMPONENT(SUP_P#,SUB_P#,...)
```

where two part number (P#) attributes are involved. It is not forbidden, however, for an attribute of one relation to have the same name as an attribute of some other relation. If this occurs, a query like that of example 3.11 could be constructed which would give rise to ambiguous naming of workspace attributes. Consider the possibility that J.JNAME and S.SNAME had been named J.NAME and S.NAME respectively (a legal possibility). A query of the form

```
GET W (J.NAME,S.NAME) ...
```

would raise the question of how the two attributes of W are to be distinguished.

To resolve this problem the name inheritance rule is formulated as follows.

When information is transmitted from the data base into a workspace W in accordance with a target list of the form

$$(R_1.D_1, R_2.D_2, \dots, R_n.D_n)$$

the jth attribute of W inherits the prefixed name $R_j.D_j$ if R_j is a relation name, and $S_j.D_j$ if R_j is a tuple variable whose range is the relation S_j . The prefix may be dropped, if the unprefix name does not occur as part of any other attribute name for that workspace.

3.13 Using a Workspace as Just Another Relation

** Find the names and locations of all suppliers who supply at least those projects supplied by supplier Jones.

One approach to formulating this query is to decompose it into two queries. First, develop as a unary relation in workspace W1 the set of project numbers supplied by supplier Jones. Then use this set to obtain the required suppliers' names and locations as a binary relation in workspace W2.

```
RANGE    SUPPLY Z
RANGE    SUPPLIER T SOME
GET W1 Z.J#: (Z.S#=T.S#)^(T.NAME='JONES')
RANGE    SUPPLIER S
RANGE    W1 X ALL
RANGE    SUPPLY Z SOME
GET W2 (S.NAME,S.LOC): (S.S#=Z.S#)^(Z.J#=X.J#)
```

When a new relation is formed in a workspace, this relation does not thereby become available to the community of users as a data base relation. Additional steps are required to make such a relation publicly available (see example 3.31).

3.14 Simple Functions in the Target List

An expandable library of functions which can be invoked in queries provides a means of extending the selective capability of DSL ALPHA.

Some rather important members of this library are the following:

COUNT	counts the number of elements in a finite set;
MAX	selects the maximum value in a finite set of numbers;
MIN	selects the minimum value in a finite set of numbers;
TOTAL	forms the sum of all <u>occurrences</u> of values in a numeric attribute (duplicates, if any, are included in the sum)

The function TOTAL has two arguments (a relation and a numeric attribute), whereas COUNT, MAX, MIN have only one (a set name or set expression).

** Find the number of suppliers who supply project #5

```
RANGE    SUPPLY Z
GET W COUNT(Z.S#):(Z.J#=5)
```

The name inheritance rule is extended to accommodate functions (simple or otherwise) in the target list. A workspace attribute formed by application of a function is given a multiply-prefixed name, the prefixes consisting of the function name and the argument names. Any tuple variable occurring as an argument is replaced by the name of its range (see 3.16). Prefixes are removed if no ambiguity results.

3.15 Simple Functions in the Qualification

We now consider examples of Boolean functions which can usefully appear in a qualification expression. Let X be a tuple belonging to relation R , let D be a numeric attribute of R and let N be an integer.

$TOP(N, X.D)$ has the value true if the D -component of tuple X has a value which is the N th largest in $R.D$ (the projection of R onto D); otherwise, its value is false.

$BOTTOM(N, X.D)$ is similarly defined, except "largest" is replaced by "smallest".

** Find the part numbers of all parts having the largest quantity-on-hand.

```
RANGE    PART P
GET W P.P#:TOP(1,P.QOH)
```

3.16 Image Functions in Target List

Consider the SUPPLY relation. The set of supplier numbers associated

under this relation with a given part number is an example of what may be called an image set. The function COUNT applied to this set would yield the number of suppliers who supply this particular part.

So common is this kind of construction and so often can the actual generation of the image set be avoided that we introduce composite functions called I-functions. For example, ICOUNT applied to a SUPPLY tuple, the P# attribute, and the S# attribute yields the count of suppliers who supply the part identified by the P# component of that SUPPLY tuple. ITOTAL applied to a SUPPLY tuple and the attributes P#, QUANTITY yields the total quantity being supplied of the part identified by the P# component of that SUPPLY tuple. IMAX, IMIN, IVERAGE correspond in an analogous way to MAX, MIN, AVERAGE.

** For each part number being supplied to a project, find as a triple the part number, the project number, and the total quantity of that part being supplied to that project.

```
RANGE      SUPPLY Z
GET  W (Z.P#,Z.J#,ITOTAL(Z,(P#,J#),QUANTITY))
```

Note that, in this query, image sets of QUANTITY values are conceptually formed for each distinct pair of values (Z.P#,Z.J#). The first argument of the function ITOTAL is Z, the second is the compound attribute (P#,J#), and the third is QUANTITY.

The name inheritance rule results in the following attribute names for W: P#,J#,QUANTITY. The prefix ITOTAL_SUPPLY_P#_J# is dropped from QUANTITY because no ambiguity results.

3.17 Image Functions in the Qualification

** Find the part number of parts supplied to more than two projects.

```
RANGE      SUPPLY Z
GET  W Z.P#:(ICOUNT(Z,P#,J#)>2)
```

Image functions can, of course, appear in both the target list and the qualification.

3.18 Piped Query

In all of the examples of queries given so far, the information requested by the user is completely assembled into the specified workspace before the user (or his application program) regains control from the data base system. If the expected amount of information is large, a correspondingly large workspace would be required to hold all of it. To conserve space the system can be requested to deliver the desired relation one element (or tuple) at a time. We refer to this as piped transmission.

Take query 3.6 for example. To obtain the response element-by-element we use an OPEN statement which specifies the details of the request.

RANGE SUPPLIER S

RANGE SUPPLY Z SOME

OPEN GET W (S.NAME,S.LOC):(S.S#=Z.S#)^(Z.P#=3)

Each succeeding execution of

GET W

causes the replacement in W of the previous pair of values of supplier name and location by a new pair of values. The order of delivery is system-determined unless an element ordering expression is included in the OPEN_GET statement. Control is returned to the user (or his application program) after each new pair of values is copied into W.

To discontinue piped transmission a statement of the form

CLOSE W

can be used, where W is the workspace involved. When the last element of qualified data has been delivered, and a further

GET W

is executed, the system returns a signal indicating the end of qualified data.

3.19 Simple Update

** Add DELTA to the quantity-on-hand QOH for the part with part number 3.

HOLD W PART.QOH:(PART.P#=3)

W.QOH = W.QOH + DELTA (host language)

UPDATE W

The HOLD statement has the same effect as the corresponding GET in regard to the information made available in the specified workspace. An additional effect of the HOLD statement is that it warns the system to be prepared to return modified data to the elements supplying the retrieved data. This returning of modified data is requested by the UPDATE statement.

Upon receipt of a HOLD the system retains enough primary key information (associating it with the particular workspace W) so that it can perform the UPDATE properly. If the value of the primary key in W has changed when the UPDATE is received, the system balks and signals an error. The system may also suspend other non-query accesses until this UPDATE is completed or cancelled. Cancellation is requested by means of

RELEASE W.

3.20 Primary Key Update

A sharp distinction is made between updates of primary keys (including components thereof) and updates of other domains, because of the crucial role of primary keys in identification and search, and the high impact on the user community of changes in these keys.

** Change the primary key of the part with the part number 3 to 4

GET W PART:(PART.P#=3)

DELETE PART:(PART.P#=3)

W.P# = 4 (host language)

PUT W PART

The GET statement retrieves the entire PART tuple which has 3 in its P# component. The DELETE statement then deletes that tuple from the data base relation PART (not from W). The host language statement alters the primary key value to 4. Finally, the PUT statement inserts this new tuple into the data base relation PART.

3.21 Multi-relation Update

The HOLD statement (unlike the GET) is restricted to a single relation occurring in its target list. This avoids unwarranted complexity. Thus multi-relation updates entail the use of more than one HOLD statement and possibly more than one workspace.

** A partial shipment of part number 3 has arrived for project number 7 from supplier number 5 with shipdate January 31, 1970. Reduce the corresponding quantity Q by 30 and increase the quantity-on-hand QOH for that part by 30.

RANGE SUPPLY Z

HOLD W Z.Q:(Z.P#=3)^(Z.S#=5)^(Z.J#=7)

^(Z.SHIPDATE=70,1,31)

W.Q = W.Q - 30 (host language)

UPDATE W

The system retains at least the full compound value of the primary key from the HOLD to the UPDATE, in order to be able to return the new values to the proper tuple.

HOLD W1 PART:(PART.P#=3)

W1.QOH = W1.QOH + 30 (host language)

UPDATE W1

3.22 Piped Update

The two transmission modes, piped and non-piped, which were described for queries are equally applicable to updates. The OPEN statement for preparing the system for a piped update is OPEN UPDATE.

** Update all SUPPLY tuples which have a SHIPDATE of January 31, 1970 on a piped basis.

OPEN UPDATE W SUPPLY:(SUPPLY.SHIPDATE=70,1,31)

HOLD W

... host language statements...

UPDATE W

Each successive execution of a HOLD, UPDATE pair causes one tuple of SUPPLY to be updated. At any instant W contains only one tuple. Whenever it is discovered that a tuple in W need not be returned to the data base, RELEASE W may be substituted for UPDATE W.

At any time CLOSE W can be used to cancel all further transmission per the corresponding OPEN statement.

3.23 All-occurrence Update

Consider the underlying domain P#. It occurs as the primary key in the relation PART, and both as a foreign key and as a component of the primary key in the relation SUPPLY. It could also occur in many other relations of the sample data base. Suppose we wish to replace all existing values of P# by a new set in accordance with a given function (tabulated or algorithmic).

One approach to this problem is first to form expanded relations in the data base -- relations in which the old part numbers and the new co-exist for a while. Later the old part numbers are dropped.

Another approach is to apply the primary key update technique of example 3.20 in turn to each data base relation that contains an attribute whose underlying domain is P#. To prepare this procedure in a general form implies a search of the catalog of relations to find the names of all relations and attributes affected by this global change. The catalog of relations can itself be a part of the data base and would then consist of additional relations. Detailed treatment of this will be postponed

to another paper. Suffice it to say that in order to make use of the names of affected relations and attributes (once these have been transmitted to a workspace), it is necessary to have in the data sublanguage an indirect reference capability. This is provided in DSL ALPHA by the PER function. For example, execution of

GET W PER(W1.RELNAME)

causes the relation, whose name is contained in W1.RELNAME, to be copied from the data base into workspace W.

3.24 Simple Insertion

** Insert into the quaternary data base relation PART the 4-tuples now located in workspace W.

PUT W PART

In all PUT statements exactly one data base relation is specified. The system checks that no duplicate values of the primary key are introduced by the PUT operation.

3.25 Partial Insertion

** Insert into the quaternary relation PART the (P#,COLOR,QOH) triples now located in workspace W.

PUT W PART.(P#,COLOR,QOH)

The attribute ordering of W is as specified in the parentheses: that is, first P#, then COLOR, then QOH. The element ordering in W may be any ordering.

The system converts each triple from W into a 4-tuple by appending the absent value as the value of PART.WEIGHT.

3.26 Ordered Insertion of Bulk Data

** Same as 3.25, except that the element ordering in W is by P# increasing, and the system is to be so advised for efficiency reasons.

PUT W PART.(P#,COLOR,QOH) UP P#

If it happens that the data base relation PART is represented in storage by a data set ordered on P#, the system can make this insertion more rapidly if advised about the ordering in W. This could be important in case the number of tuples to be inserted is large. Such a case often occurs when a new relation is being established (see example 3.31).

3.27 Piped Insertion

The piped mode of transmission can be applied to insertion just as to queries.

** Same as 3.26, except that transmission is to be in the piped mode.

```
OPEN PUT W PART.(P#,COLOR,QOH) UP P#  
... host language statements ...  
  
PUT W  
... host language statements ...
```

Each successive execution of a PUT statement causes a tuple from W to be inserted into the data base relation PART. At any instant W contains only one tuple. As usual the piped transmission from W can be terminated by execution of

```
CLOSE W.
```

3.28 Simple Deletion

** Delete the tuple for part number 5 from the data base relation PART

```
DELETE PART:(PART.P#=5)
```

No workspace is required as an information sink, and this applies to all deletions. Normally, none is needed as an information source, but see example 3.30.

3.29 Qualified Deletion

** Delete all tuples of the data base relation SUPPLY involving supplier JONES and project ALPHA in combination with one another.

```
RANGE SUPPLY Z  
RANGE SUPPLIER S SOME  
RANGE PROJECT J SOME  
DELETE Z:(S.NAME='JONES')^(S.S#=Z.S#)  
^ (Z.J#=J.J#)^(J.NAME='ALPHA')
```

A single DELETE statement removes tuples from precisely one data base relation (no more, no less). The qualification in a DELETE can involve any number of relations and quantifiers.

3.30 Workspace-Conditioned Deletion

** Delete those tuples of the data base relation SUPPLY whose part numbers are contained in workspace W2.

DELETE SUPPLY:(SUPPLY.P#=W2.P#)

The workspace W2 used in this deletion acts only as a source of conditioning information.

3.31 Establishing a New Data Base Relation

Suppose a new relation has been developed in workspace W and it is desired that this relation be established in the data base for shared use. Before the data can be copied from W into the data base, several actions must be taken:

1. The relation to be established must now be declared. This involves giving it a name which is not in conflict with those of other data base relations, naming its domains (if not already declared) and attributes in accordance with community-established rules, identifying its primary key (possibly a compound attribute), and giving certain characteristics of the components of its tuples. All of this information must be entered in those relations which catalog the data base relations.
2. The access authorization constraints for the new relation must be set up in those relations which describe these constraints.
3. A storage representation must be selected for the new relation (this may include a decision as to which attributes are to be indexed) and this descriptive data must be stored in appropriate relations.

When these preparatory "gold-plating" steps have been taken (and they will be dealt with more fully in a later paper), the relation is ready to "go public". A statement of the form

PUT W R

then sets up the initial data for the new relation R.

The following declarations describe the underlying domains for the sample data base used throughout this section:

DOMAIN S# CHAR(5)
SNAME CHAR(15)
LOC CHAR(20)
P# CHAR(20)

PNAME CHAR(30)
 COLOR STATE(1='BLUE',2='RED',...)
 WEIGHT NUM(3,1)
 QOH NUM(4,0)
 J# CHAR(4)
 JNAME CHAR(10)
 E# NUM(6,0)
 SHIPDATE DATE(YEAR,MONTH,DAY)
 QUANTITY NUM(3,0)

Data base relations are declared separately from the underlying domains, so that a new relation may be readily defined on already declared domains. Included in each relation declaration is identification of the primary key. The following declarations describe the relations in the sample data base under consideration:

RELATION SUPPLIER(S#,SNAME,LOC,...) KEY S#
RELATION PART(P#,PNAME,COLOR,WEIGHT,QOH) KEY P#
RELATION PROJECT(J#,JNAME,MGR_E#,...) KEY J#
RELATION SUPPLY(S#,P#,J#,SHIPDATE,QUANTITY)
 KEY (S#,P#,J#,SHIPDATE)

Note the attribute name MGR_E# has the prefix MGR and stem E#. The underlying domain is indicated by its stem E#. In all other cases the attribute name is identical with the name of its underlying domain.

3.32 Dropping a Data Base Relation or Domain

If R is the name of a data base relation, execution of the statement

DROP R

has the effect of removing all information about R from the data base catalog. Further, if any tuples of R still exist, they are deleted. Note that declarations of the underlying domains on which R is defined remain in force.

Information about an underlying domain D may be removed from the data base catalog by execution of the statement

DROP D

providing there is no data base relation defined on D. Any attempt to drop a domain still in use by some relation is treated as an error.

The statement `DROP R` (where R is a relation) should be contrasted with `DELETE R` which merely deletes all tuples of R, but leaves R as an established (albeit empty) relation of the data base.

3.33 Other Operations and Attributes

Several additional features are desirable (in some cases necessary) to handle interlocking (for example, a means whereby certain operations may be temporarily excluded from specified portions of the data base), access authorization, integrity preservation, virtual attributes, literal insertions, etc. Various types of errors and the feedback which a user might expect from them have also been intentionally omitted. These topics will be discussed in a later paper.

4. Comparison of Language Levels

In this section we consider the pros and cons of the three principal levels of data base language identified in the introduction. Arguments in favor of a relational model of the data were presented earlier by this author (see [1]). Except for the first point below, these arguments are not repeated here, but it should be noted that they strongly re-inforce the arguments for both the intermediate algebraic and the high level calculus-oriented languages.

4.1 Protection of Users from Representation Clutter

The question of providing the majority of users with a conceptually concise model of the data and a powerful, conceptually concise language for its manipulation is not just an aesthetic concern. When users are forced to make numerous choices and decisions about avoidable representation details, the consequences are manifold and costly. Programs inevitably become more elaborate, hence more time-consuming to write, to debug, and to modify. This is not merely an argument for protecting users from manipulating the sordid details of the physical (or storage) representation of the data. It is an equally valid argument against imposing on the user an over-elaborate, conceptually redundant logical data representation. Perhaps the most elaborate data model is to be found in the CODASYL proposal [4]. The Data Base Task Group "set" concept together with all its attendant ramifications could be dropped without losing any logical power in storage and retrieval. The algebraic and calculus-oriented data sublanguages score heavily in protecting users from representation clutter by using a simple data model and by treating relations as data items manipulable in their entirety. The procedural approach, on the other hand, treats relations or sets in an element-by-element and connection-by-connection fashion.

4.2 Descriptive versus Constructive Expression of Intent

In the low-level procedural approach much of the searching for data is hand-coded by the application programmer. In the relational algebraic approach more of the responsibility for searching is taken over by the data base system. Finally, in the relational calculus approach the system takes over complete responsibility for searching.

From the linguistic standpoint we can characterize the procedural and algebraic approaches as constructive, while the calculus approach is descriptive. The clarity of intent of the descriptive approach facilitates all those system activities which depend on discovery by the system of the user's intent--for example:

- optimal search
- authorization
- transformability in case of structural change in the data model.

4.3 Understanding and Modifying Programs

As a corollary of points 1 and 2, clarity of intent is important for programs to be understandable by people, especially those who did not write the programs. This becomes of crucial importance when an application program has to be changed because the application itself has changed.

As a simple example of modifiability of programs, compare the work needed to change the order of two quantifiers in a DSL ALPHA program with the work needed to re-structure a corresponding CODASYL program to give the same effect.

4.4 Evolutionary Development of Search Techniques

As a further corollary of point 2, adoption of the calculus-oriented approach permits successive improvements in general search algorithms to be incorporated into data base systems without impacting users' programs (the language in which these programs are written stays constant).

4.5 Evolutionary Development of Data Structures

By adopting a data model of reasonable abstraction (that is, not too close to the currently fashionable physical representations) more scope is provided for future implementations to evolve in the data organizations and physical representations which they use to support the data model. This evolution can take place without logically impairing users' application programs.

In this regard, both types of relational languages (the algebraic and the calculus-oriented) are superior to the network procedural type.

4.6 Support for Specialized Query-Update Languages

Many users need query languages specialized to their applications. The high cost of supporting a great variety of these languages with translators suggests that as many as possible of the common services in these translators be identified and programmed once and for all. This, in turn, suggests adopting as a target language for these translators a very high level storage and retrieval language. The calculus-oriented type of language appears best suited for this purpose. Indeed, research done to-date in the relatively free use of natural language, such as English, for storage and retrieval purposes suggests that the calculus-oriented language is an appropriate stepping stone toward this goal.

5. Summary

In section 1 three types of language for data base manipulation were identified. Information on two of these (the low-level procedural and the intermediate algebraic) is available in the literature. An example of the third type (based on the relational calculus) is described in sections 2 and 3. The comparison of these three types in section 4 suggests that more attention be paid to the relational calculus type before computer users, the computing industry and society in general get committed to the heavy costs of implementing proposed data base language standards and to the long-term binding of their application programs to these standards.

6. Acknowledgments

An early draft of this paper was circulated under the title "Notes on a Data Sublanguage". It is a pleasure to acknowledge the benefit of considerable feedback since then, particularly from T. R. Heywood, C. J. Date and P. Hopewell of the IBM Laboratory in England, from Sharon Weinberg and Philip Shaw of the New York Programming Center, and from Erich Neuhold of the Vienna Laboratory.

The author is also grateful to Robert W. Engles of Poughkeepsie and Robert L. Griffith of San Jose for suggesting changes which improved the clarity.

References

1. E. F. Codd, "A Relational Model of Data for Large Shared Data Banks", Comm. ACM 13 6 June 1970, 377-387.
2. E. F. Codd, "Further Normalization of the Data Base Relational Model", Courant Computer Science Symposia 6 "Data Base Systems", to be published by Prentice-Hall, May 1971.
3. E. F. Codd, "Relational Completeness of Data Base Sublanguages", Courant Computer Science Symposia 6 Data Base Systems", to be published by Prentice-Hall, May 1971.
4. CODASYL "Data Base Task Group Report", ACM HQ, October 1969.
5. Joint Guide-Share, "Data Base Management System Requirements", Guide or Share Distribution, November 1970.
6. Information Management System IMS/360, Application Description Manual (Version 2) GH20-0765-1 IBM, White Plains, New York, 1971.
7. Generalized Information System GIS/360, Application Description Manual (Version 2) GH20-0892-0 IBM, White Plains, New York, 1970.
8. T. W. Olle, "A Non-procedural Language for Retrieving Information from Data Bases", IFIP Congress, Edinburgh, August 1968, North-Holland, Amsterdam.
9. IDS Reference Manual GE625/635, GE Information Systems Division, Phoenix, Arizona, CPB 1093B, February 1968.
10. J. A. Feldman, P. D. Rovner, "An Algol-Based Associative Language", Comm. ACM 12, 8 August 1969, 439-449.
11. W. Ash, E. H. Sibley, "TRAMP, an interpretive associative processor with deductive capabilities", Proc ACM 23rd Nat. Conf., Brandon Systems Press, Princeton N.J., 1968, 143-156.
12. A. N. Whitehead, B. Russell, "Principia Mathematica", Vol 1., Cambridge University Press, 1960.
13. R. E. Levien, M. E. Maron, "A Computer System for Inference Execution and Data Retrieval", Comm. ACM 10, 11 November 1967, 715-721.
14. C. J. Bell, "A Relational Model for Information Retrieval and the Processing of Linguistic Data", IBM Research Report RC1705, Yorktown Heights, New York, November 1966.
15. D. L. Childs, "Description of a Set-Theoretic Data Structure", Proc. FJCC, Vol 33, Part 1, December 1968, 557-564.

16. R. C. Goldstein, A. L. Strnad, "The MacAIMS Data Management System" Proc. 1970 ACM-SICFIDET Workshop on Data Description and Access", ACM HQ, November 1970.
17. A.L. Strnad, "The Relational Approach to the Management of Data Bases", to be presented at IFIP Congress, Ljubljana 1971.
18. D. Cantor, B. Dimsdale, A. Hurwitz, "Query Language One (QL/1) Users Manual", Report 320-2627, IBM Scientific Center, Los Angeles, Calif., June 1969, 15-19.
19. E. S. Lowry, "Proposed Language Extensions to Aid Coding and Analysis of Large Programs", IBM Poughkeepsie Laboratory Technical Report TR00.1940-1, November 1969.
20. R. E. Bleier, "Treating Hierarchical Data Structures in the SDC Time Shared Data Management System TDMS, Proc. ACM 22nd Nat. Conf., MDI Publications, Wayne, Pa, 1967, 41-49.

Additional Bibliography

21. P. Suppes, "Introduction to Logic", van Nostrand, 1967.
22. R. W. Engles, "A Tutorial on Data Base Organization", IBM Poughkeepsie Laboratory Technical Report TR00.2004, March 1970.
23. E. J. Neuhold, "Data Mapping: A Formal Hierarchical and Relational View", Courant Computer Science Symposia 6, "Data Base Systems", to be published by Prentice-Hall, May 1971.
24. R. A. Lorie, A. J. Symonds, "A Schema for Describing a Relational Data Base", Proc. 1970 ACM-SICFIDET Workshop on Data Description and Access, ACM HQ, November 1970.
25. C. J. Date, P. Hopewell, "File Definition and Logical Data Independence", Proc. 1971 ACM-SIGFIDET Workshop on Data Description, Access and Control, to be available from ACM HQ, 1972.
26. C. J. Date, P. Hopewell, "Storage Structure and Physical Data Independence", Proc. 1971 ACM-SIGFIDET Workshop on Data Description, Access and Control, to be available from ACM HQ, 1972.
27. I. J. Heath, "Unacceptable File Operations in a Relational Data Base", Proc. 1971 ACM-SIGFIDET Workshop on Data Description Access and Control, to be available from ACM HQ, 1972.

APPENDIX: DEFINITIONS

The term relation is used in this paper in its accepted mathematical sense. Given sets $S_1, S_2, \dots, S_n$ (not necessarily distinct), R is a relation on these n sets if it is a set of n -tuples each of which has its first element from S_1 , its second element from S_2 , and so on. More concisely, R is a subset of the Cartesian product $S_1 \times S_2 \times \dots \times S_n$. We refer to S_j as the j th domain of R . As defined above, R is said to have degree n . Relations of degree 1 are often called unary, degree 2 binary, degree 3 ternary, and degree n n -ary.

Relations are sets, but not all sets are relations. The elements of a relation of degree n are called n -tuples or tuples. A relation is in first normal form if it has the property that none of its domains has elements which are themselves sets. An unnormalized relation is one which is not in first normal form.

A candidate key K of relation R is a combination of attributes* (possibly a single attribute) of R with properties P_1 and P_2 :

- P_1 : (Unique identification) In each tuple of R the value of K uniquely identifies that tuple;
- P_2 : (Non-redundancy) No attribute in K can be discarded without destroying property P_1 .

For each relation R in a data base, one of its candidate keys is arbitrarily designated as the primary key of R . An attribute or attribute combination of R is a foreign key if it is not a candidate key of R but its elements are values of the primary key of some relation S (the possibility that R and S are identical is not excluded).

* See section 2.2.